Atividade 3 - Elicitação e Priorização de Requisitos

Projeto: Verdigris: Motor de Decisão Coletiva
Autor: Felipe Leme de Argollo Ferrão (RA: 260431)

1. Elicitação de Requisitos

Técnica Escolhida e Justificativa

- **Técnica:** Brainstorming individual baseado em Personas e Cenários (Role-playing)
- **Justificativa:** Como o desenvolvimento do Verdigris ocorre em formato solo e com severas restrições de tempo de desenvolvimento, agendar e conduzir entrevistas com usuários reais atrasaria o cronograma técnico. O brainstorming guiado por personas permite que o desenvolvedor mude o "chapéu" e adote as perspectivas dos diferentes stakeholders do motor (administrador da eleição, eleitor e auditor) de forma assíncrona e ágil.

Fontes e Participantes

- **Participantes/Fontes:**
  - Felipe Leme (desenvolvedor solo).
  - Personas simuladas:
    1. Carlos (Presidente de Centro Acadêmico)
    2. Sônia (Síndica de Condomínio)
    3. Alex (Eleitor/Auditor focado em privacidade)

Condução da Atividade

- Foi realizado um exercício de *time-boxing* de 30 minutos utilizando um quadro virtual em branco. Para cada persona, o desenvolvedor simulou o fluxo de criar uma eleição, votar e apurar os resultados, respondendo à pergunta-chave: "O que me faria desistir de usar esse motor e voltar para o papel/Google Forms?". As ideias e objeções foram registradas em notas virtuais (post-its) e agrupadas por afinidade temática.

Principais Descobertas

- **Necessidade de Apuração Injetável (Custom Tally):** Na simulação da síndica e do presidente do CA, ficou claro que as regras de apuração mudam muito dependendo do estatuto vigente. *Oportunidade técnica:* Em vez de limitar o sistema a um enum de estratégias fixas, o motor precisa suportar uma CustomElection onde o responsável possa injetar regras de peso e contagem dinamicamente (passando funções personalizadas/closures como parâmetro para o tally).

- **Quórum Híbrido:** Em condomínios, o quórum não é contado apenas por "número de cabeças", mas pela soma das frações ideais das propriedades presentes. O sistema precisa separar o conceito de "quantidade de votos" de "peso acumulado".
- **Comprovante Cego:** O eleitor (Alex) exige um recibo para auditar se o seu voto está na urna, mas esse recibo não pode revelar a escolha do voto para evitar coerção ou compra de votos.
- **Isolamento de Identidade:** O administrador da eleição precisa saber quem já votou para não permitir votos duplos, mas a urna só deve armazenar o conteúdo do voto, quebrando o vínculo criptográfico entre os dois momentos.

Evidências

As notas abaixo representam a transcrição do quadro virtual gerado durante a sessão.

> ***[Board - Perspectiva do Administrador]***
> *- "E se a regra de peso mudar no meio do ano? Preciso conseguir plugar minha própria fórmula matemática de apuração." → Ação: Criar arquitetura que suporte Tally functions injetáveis.*
> *- "Tenho que provar para os condôminos que não alterei os votos." → Ação: O banco de dados (ledger) não pode ter função de UPDATE/DELETE.*

> ***[Board - Perspectiva do Eleitor/Auditor]***
> *- "Como sei que meu voto no candidato Y foi para a urna certa?" → Ação: Gerar hash no momento do voto (Recibo).*
> *- "Se o administrador cruzar a lista de presença com a urna, ele descobre em quem eu votei?" → Ação: Arquitetura deve emitir um 'token de elegibilidade' cego. A urna só conhece o token, não o CPF do aluno.*

2. Da Elicitação aos Requisitos

As descobertas obtidas durante a simulação de cenários orientaram diretamente a arquitetura do motor Verdigris. Foi identificado um conflito central durante a elicitação: a necessidade administrativa de garantir que um eleitor não vote duas vezes (controle de presença) versus a exigência do eleitor de que seu voto seja absolutamente sigiloso. A equipe decidiu tratar esse conflito dividindo a arquitetura em dois fluxos isolados: um responsável pela verificação de elegibilidade (que emite um token cego) e outro responsável por receber o voto (a urna, que valida o token sem saber quem o gerou).

A tabela a seguir relaciona as descobertas aos requisitos e épicos mapeados:

Descoberta da elicitação	Análise / interpretação	Requisito relacionado
Apuração Injetável: Diferentes contextos (CA, Condomínio) possuem regras que podem mudar. O administrador precisa plugar fórmulas próprias.	O motor não pode ter estratégias hardcoded. Deve expor uma interface (traits/closures) para injeção de regras de contagem dinâmicas na criação da eleição.	**RF - Configuração de Apuração Customizada** (Épico #13)
Quórum Híbrido: Em condomínios, a validade não se dá pelo número de pessoas, mas pela soma das frações ideais.	A lógica de estado da eleição precisa desvincular "quantidade de cédulas" do "peso total acumulado" para validar encerramentos.	**RF - Validação de Quórum Qualificado** (Épico #13)
Comprovante Cego: O eleitor quer auditar se seu voto foi contabilizado sem revelar em quem votou.	É necessário gerar um identificador criptográfico único (hash) por voto.	**RF - Emissão de Recibo de Auditoria** (Épico #15)
(Continuação do comprovante cego)	A emissão do recibo não pode comprometer o sigilo sob nenhuma circunstância.	**RNF - Privacidade do Voto:** O hash do recibo gerado não deve permitir engenharia reversa para descobrir o conteúdo do voto. *Transversal ao Épico #15.*
Isolamento de Identidade: Risco de cruzar listas de presença com a ordem dos votos na urna.	A urna deve apenas armazenar eventos lógicos e rejeitar qualquer dado que identifique o eleitor (ex: CPF, RA).	**RF - Separação do Registro de Elegibilidade** (Épico #14)
(Continuação do isolamento)	Para garantir a confiança na separação, os componentes devem ser sistemicamente desacoplados.	**RNF - Isolamento de Dados:** Nenhuma estrutura de dados do núcleo de apuração (EventStore) pode conter campos que armazenem identificadores pessoais diretos. *Transversal ao Épico #14.*

3. Épicos, Histórias e Critérios de Aceitação

Épico #13: Núcleo de Apuração Customizável

Descrição: Desenvolver o motor base (núcleo comum) que permite a execução de diferentes estratégias de apuração de votos (maioria simples, ranqueamento de Borda, voto ponderado). O objetivo é garantir que o mesmo motor possa ser reutilizado em múltiplos contextos (Centro Acadêmico, Condomínios e Orçamento Participativo) sem necessidade de reescrever a base lógica da eleição.

História	**#16 - Configurar estratégia de apuração injetável**
Descrição	**Como** administrador de eleição, **quero** injetar regras de apuração e limites customizados na criação da eleição, **para** reaproveitar o mesmo motor de votação em contextos diferentes, como CA e Condomínios.
Critérios de Aceitação	1. O sistema deve aceitar uma abstração de estratégia (TallyStrategy) no momento de instanciar a eleição (Builder pattern). 2. A eleição deve rejeitar a criação e retornar um erro se os parâmetros da estratégia estiverem vazios (ex: zero candidatos cadastrados). 3. O núcleo genérico não deve sofrer alterações em seu código-fonte ao receber uma nova estratégia de apuração.
Relacionamento	Pertence ao Épico #13

História	**#17 - Validar quórum qualificado por peso**
Descrição	**Como** síndica de condomínio, **quero** que a eleição valide o quórum com base na soma das frações ideais das propriedades, **para** garantir que as decisões respeitem o estatuto do condomínio, que exige um percentual mínimo de área representada.
Critérios de Aceitação	1. O voto submetido (CondoVote) deve conter e processar a fração da propriedade (peso). 2. O encerramento da eleição deve retornar a falha "Quórum não atingido" se a soma de todas as frações for menor que o limite definido na criação. 3. A estratégia deve rejeitar votos cuja fração seja menor ou igual a 0.0 ou maior que 1.0.
Relacionamento	Pertence ao Épico #13

Épico #14: Elegibilidade e Sigilo

Descrição: Construir a arquitetura que separa o controle de presença (quem tem o direito de votar) da submissão da cédula, evitando votos duplos e garantindo que o voto permaneça anônimo.

História	**#21 - Emissão isolada de token de elegibilidade**
Descrição	**Como** presidente do Centro Acadêmico, **quero** que o sistema libere tokens de acesso apenas para alunos ativamente matriculados que ainda não votaram, **para** garantir a unicidade do voto e a legitimidade dos eleitores.
Critérios de Aceitação	1. O sistema deve validar a identidade informada contra a lista de eleitores cadastrados na eleição. 2. Se elegível, o eleitor é marcado como "já atendeu" (presença confirmada) e recebe um token de uso único. 3. Se o eleitor já solicitou um token antes, a emissão deve ser bloqueada.
Relacionamento	Pertence ao Épico #14

História	**#20 - Submissão de voto anônima por token**
Descrição	**Como** eleitor focado em privacidade, **quero** que a urna aceite meu voto apenas mediante a apresentação de um token válido e desvinculado, **para** que meu registro acadêmico não seja criptograficamente ou logicamente ligado à minha cédula.
Critérios de Aceitação	1. O endpoint de submissão da urna deve aceitar apenas o token de autorização e a escolha (conteúdo do voto), rejeitando campos de identificação pessoal (como RA). 2. A urna deve invalidar o token imediatamente após o uso bem-sucedido. 3. O banco de dados da urna e o banco de elegibilidade não devem compartilhar chaves estrangeiras.
Relacionamento	Pertence ao Épico #14

Épico #15: Integridade e Auditoria

Descrição: Implementar os mecanismos de persistência e validação criptográfica que garantem que os votos não possam ser adulterados após submetidos, e fornecer as ferramentas necessárias para que os eleitores auditem publicamente a eleição.

História	**#18 - Registro de eventos append-only**
Descrição	**Como** auditor de sistemas, **quero** que os eventos da eleição e os votos sejam persistidos em um registro estritamente incremental (append-only), **para** evitar que votos sejam apagados ou modificados no banco de dados após sua submissão.

Critérios de Aceitação	1. A interface do repositório (EventStore) deve conter apenas o método de adição (append/insert), sem expor update ou delete. 2. Cada evento deve receber um número de sequência estritamente crescente. 3. Se houver tentativa de inserção concorrente com o mesmo número de sequência, o sistema deve rejeitar o segundo evento com erro de conflito de concorrência.
Relacionamento	Pertence ao Épico #15

História	**#19 - Emissão de recibo de auditoria cego**
Descrição	**Como** eleitor, **quero** receber um comprovante único (hash) da submissão do meu voto, **para** poder auditar publicamente se minha participação foi contabilizada na urna sem revelar qual foi a minha escolha.
Critérios de Aceitação	1. Ao submeter um voto válido na API, o sistema deve retornar ao usuário um identificador único irreversível (hash). 2. O hash gerado não pode ser reconstruído para revelar a identidade do eleitor ou do candidato escolhido (cédula opaca). 3. O sistema deve permitir a listagem de todos os hashes presentes na urna após o encerramento da eleição.
Relacionamento	Pertence ao Épico #15

4. Organização e Priorização do Backlog

Organização no GitHub Projects: O backlog foi estruturado em um quadro Kanban (Board view) padrão no GitHub Projects, contendo as colunas Todo, In Progress e Done. A priorização foi estabelecida através da ordenação vertical dos itens na coluna Todo, onde os cards no topo representam a maior prioridade de execução. Os épicos e as histórias foram interligados utilizando as referências de issues (ex: #16 referenciando #13).

Primeira Entrega de Valor: A primeira entrega de valor tem como objetivo viabilizar uma eleição funcional e segura para o Centro Acadêmico (MVP). Para que essa entrega seja um conjunto coerente, ela precisa ser capaz de: instanciar uma eleição com regras básicas, emitir tokens de acesso para garantir unicidade, receber votos anonimamente e persistir esses votos de forma segura (append-only). Funcionalidades avançadas, como quórum qualificado por peso (Condomínios) e recibos cegos de auditoria, foram priorizadas para iterações seguintes, pois não bloqueiam o fluxo principal de votação.

Priorização dos Épicos

Prioridade	Épico	MVP?	Justificativa
1	**#13 - Núcleo de Apuração Customizável**	Sim	É o motor central da aplicação. Sem a capacidade de instanciar a eleição e injetar regras de apuração, nenhuma outra funcionalidade pode ser executada.
2	**#14 - Elegibilidade e Sigilo**	Sim	É fundamental para garantir que uma eleição tenha valor no mundo real, prevenindo votos duplos e garantindo que o voto seja secreto desde o primeiro uso.
3	**#15 - Integridade e Auditoria**	Parcialmente	A persistência imutável dos eventos é essencial para a primeira entrega, mas os recursos avançados de auditoria pública e recibos não bloqueiam o fluxo básico e podem ser entregues depois.

Priorização das Histórias: Épico #13 (Núcleo)

Prioridade	História	MVP?	Justificativa
1	**#16 - Configurar estratégia de apuração injetável**	Sim	Permite que o sistema crie a eleição básica de contagem simples (requisito do CA) e pavimenta a arquitetura do motor.
2	**#17 - Validar quórum qualificado por peso**	Não	Apenas o contexto de Condomínio utiliza peso fracionário. A primeira entrega será validada no contexto do CA (um voto, um aluno), portanto não é bloqueante.

Priorização das Histórias: Épico #14 (Elegibilidade e Sigilo)

Prioridade	História	MVP?	Justificativa
1	**#20 - Submissão de voto anônima por token**	Sim	É o endpoint principal do sistema: receber o voto na urna desvinculado da identidade do eleitor.
2	**#21 - Emissão isolada de token de elegibilidade**	Sim	Pré-requisito estrito da história #20. Sem emitir o token, a urna não tem como autorizar o eleitor anonimamente.

Priorização das Histórias: Épico #15 (Integridade e Auditoria)

Prioridade	História	MVP?	Justificativa
1	**#18 - Registro de eventos append-only**	Sim	É a base de dados do sistema. O núcleo da urna precisa persistir a submissão dos votos de forma imutável desde a versão inicial.
2	**#19 - Emissão de recibo de auditoria cego**	Não	Adiciona valor de transparência ao eleitor, mas a urna já pode funcionar e apurar votos confiavelmente sem emitir o hash de verificação no MVP.

Snapshot do GitHub Projects

(Inserir print do board do GitHub Projects nesta área antes da entrega final)